

中南大學
CENTRAL SOUTH UNIVERSITY

操作系统原理
实验报告

学生姓名	宣丽娜
学　　号	8210241914
专业班级	计科 T2401
指导教师	胡小龙
学　　院	计算机学院
完成时间	2026-6-28

目 录

一、实验目的 4
二、实验环境 5
三、实验内容 6
四、实验原理 7
4.1 多进程创建原理 7
4.2 共享内存通信原理 7
4.3 信号量同步与互斥原理 7
4.4 管道通信原理 7
五、系统总体设计 8
5.1 系统功能设计 8
5.2 任务处理流程 8
5.3 任务队列设计 9
六、程序实现 10
6.1 主要数据结构 10
6.2 父进程实现 10
6.3 子进程实现 11
6.4 同步互斥实现 12
七、实验步骤 13
八、实验结果 14
8.1 单进程运行结果 14
8.2 四进程运行结果 15
8.3 实验结果对比 17
九、结果分析 18
9.1 共享队列生产和消费次数分析 18
9.2 多进程并发优势分析 18
9.3 同步互斥机制分析 18
9.4 进程间通信机制分析 18
十、实验中遇到的问题及解决方法 20

10.1 初始任务规模较小导致多进程优势不明显 20
10.2 共享队列访问需要避免数据竞争 20
10.3 子进程退出需要结束任务 20
十一、实验总结 21
附录：主要源程序 22

一、实验目的

本实验通过设计并实现一个 Linux 环境下的多进程并发任务处理系统，加深对操作系统中进程创建、进程间通信、同步互斥和并发执行机制的理解。实验以素数统计任务为处理对象，通过父进程创建多个工作子进程并发处理任务，观察不同进程数量下程序运行效率的变化。

通过本实验，能够掌握 fork() 创建进程的方法，理解共享内存、信号量、管道等进程间通信机制的基本使用方法，并能够分析多进程并发处理在计算密集型任务中的性能优势。

二、实验环境

项目	内容
操作系统	Linux
编程语言	C 语言
编译器	GCC
主要系统调用	fork、pipe、waitpid、shm_open、mmap、sem_open
实验类型	设计类实验

三、实验内容

本实验设计了一个基于生产者—消费者模型的多进程任务处理系统。父进程作为生产者，负责产生多个计算任务，并将任务放入共享内存中的循环任务队列；多个子进程作为消费者，从共享任务队列中取出任务并进行计算。任务内容为统计指定范围内的素数个数。

为了保证多个进程安全访问共享任务队列，程序使用信号量实现同步与互斥。同时，子进程完成任务后，通过管道将计算结果返回给父进程。父进程接收所有计算结果后，统计程序总运行时间，并输出实验结果。

本实验主要体现了进程创建、进程间通信、同步互斥、父子进程协同执行和多进程并发性能分析等操作系统核心内容。

四、实验原理

4.1 多进程创建原理

Linux 系统中可以使用 fork() 系统调用创建子进程。调用 fork() 后，操作系统会复制当前进程，生成一个新的子进程。父进程和子进程会从 fork() 返回的位置继续执行，但返回值不同。本实验中，父进程通过循环调用 fork() 创建多个工作进程，每个工作进程负责从共享任务队列中取出任务并执行计算。

4.2 共享内存通信原理

共享内存是一种高效的进程间通信方式。多个进程可以将同一块物理内存映射到各自的虚拟地址空间中，从而实现数据共享。本实验中，父进程和所有子进程共享一个任务队列，父进程将任务写入共享队列，子进程从共享队列中读取任务。

4.3 信号量同步与互斥原理

由于多个进程会同时访问共享任务队列，如果不进行控制，可能会出现数据竞争问题。例如，两个子进程可能同时读取同一个任务，或者父进程和子进程同时修改队列指针，导致队列状态错误。因此，本实验使用信号量对共享队列进行同步与互斥控制。

信号量	作用
sem_empty	表示任务队列中的空闲位置数量，防止队列满时继续写入
sem_full	表示任务队列中的可用任务数量，防止队列空时读取
sem_mutex	保护共享队列临界区，保证同一时刻只有一个进程修改队列

4.4 管道通信原理

管道是 Linux 中常见的进程间通信机制，适合具有亲缘关系的进程之间传递数据。本实验中，子进程完成素数统计任务后，将任务编号、计算范围、素数个数、执行时间等结果写入管道，父进程从管道中读取所有子进程返回的结果。

五、系统总体设计

5.1 系统功能设计

系统由一个父进程和若干个工作子进程组成。父进程负责初始化 IPC 资源、创建子进程、产生任务、投放任务、接收结果以及释放资源；子进程负责从共享队列中获取任务、执行素数统计并通过管道返回结果。

模块	主要功能
父进程模块	初始化共享内存和信号量，创建工作进程，投放任务，接收结果，回收子进程
共享队列模块	以循环队列形式保存任务，实现任务缓存和分发
工作进程模块	从共享队列获取任务，执行素数统计，返回计算结果
同步互斥模块	通过三个信号量保证队列访问正确
结果回传模块	通过匿名管道将子进程计算结果发送给父进程

5.2 任务处理流程

```
父进程 Parent
 │
 ├─ 初始化共享内存、信号量和管道
 │
 ├─ fork() 创建多个工作子进程
 │
 ├─ 产生素数统计任务并写入共享任务队列
 │
共享内存任务队列  ← sem_empty / sem_full / sem_mutex 控制同步互斥
 │
 ├─ Worker1 取任务并计算
 ├─ Worker2 取任务并计算
 ├─ Worker3 取任务并计算
 └─ Worker4 取任务并计算
 │
 └─ 子进程通过 pipe 返回结果
 │
父进程汇总结果、统计耗时、释放资源
```

图 1 系统总体流程示意图

5.3 任务队列设计

共享任务队列采用循环队列结构，队列中保存多个任务。队列包含 queue、in、out、produced 和 consumed 等成员。父进程写入任务时移动 in 指针，子进程读取任务时移动 out 指针。由于队列长度有限，因此采用循环队列方式，当指针到达队尾后重新回到队首。

六、程序实现

6.1 主要数据结构

程序中定义了 task_t、shared_queue_t 和 result_t 三个主要结构体。task_t 用于保存任务编号和计算上限；shared_queue_t 用于保存共享任务队列及队列指针；result_t 用于保存子进程返回给父进程的计算结果。

```
typedef struct {
    int id;
    int number;
} task_t;


typedef struct {
    task_t queue[QUEUE_SIZE];
    int in;
    int out;
    int produced;
    int consumed;
} shared_queue_t;


typedef struct {
    int worker_id;
    int task_id;
    int number;
    int prime_count;
    long elapsed_ms;
} result_t;
```

6.2 父进程实现

父进程首先创建共享内存，并将共享内存映射到进程地址空间。然后创建三个信号量，分别用于表示队列空位、队列任务数量以及互斥访问。父进程通过 fork() 创建多个子进程，之后开始产生任务并写入共享任务队列。所有正式任务投放完成后，父进程还会为每个工作进程投放一个结束任务，用于通知子进程退出。

6.3 子进程实现

子进程启动后，会不断从共享任务队列中获取任务。如果获取到普通计算任务，则执行素数统计，并将结果通过管道写回父进程；如果获取到任务编号为 -1 的结束任务，则说明任务已经全部处理完毕，子进程退出。

6.4 同步互斥实现

父进程写入任务时，先等待 sem_empty，确保队列有空位；再等待 sem_mutex，进入临界区；写入任务并修改 in 指针后释放 sem_mutex，最后释放 sem_full，通知子进程队列中已有任务。

子进程读取任务时，先等待 sem_full，确保队列中有任务；再等待 sem_mutex，进入临界区；读取任务并修改 out 指针后释放 sem_mutex，最后释放 sem_empty，通知队列出现空位。该机制保证了共享队列访问的正确性。

七、实验步骤

1. 在 Linux 环境下新建源文件 multiprocess_ipc.c，并输入实验程序代码。

2. 使用 GCC 编译程序，编译命令为：gcc multiprocess_ipc.c -o multiprocess_ipc -pthread -lrt。

3. 运行单进程实验，命令为：./multiprocess_ipc 20 1。

4. 运行四进程实验，命令为：./multiprocess_ipc 20 4。

5. 记录两组实验的任务数量、工作进程数、共享队列生产次数、共享队列消费次数和总耗时。

6. 对比单进程和多进程运行结果，分析多进程并发处理效果。

八、实验结果

8.1 单进程运行结果

当任务数量为 20，工作进程数量为 1 时，程序运行结果如下：

```
xln@xln-VMware-Virtual-Platform:~$ nano multiprocess_ipc.c
xln@xln-VMware-Virtual-Platform:~$ gcc multiprocess_ipc.c -o multiprocess_ipc -pthread -lrt
xln@xln-VMware-Virtual-Platform:~$ ./multiprocess_ipc 20 1
========== 多进程 IPC 实验 ==========
任务数量：20
工作进程数量：1
队列容量：8
====================================
[Parent] 投放任务 1：统计 1 到 486651 的素数个数。
[Parent] 投放任务 2：统计 1 到 425457 的素数个数。
[Parent] 投放任务 3：统计 1 到 336876 的素数个数。
[Parent] 投放任务 4：统计 1 到 316460 的素数个数。
[Parent] 投放任务 5：统计 1 到 359361 的素数个数。
[Parent] 投放任务 6：统计 1 到 499607 的素数个数。
[Parent] 投放任务 7：统计 1 到 484846 的素数个数。
[Parent] 投放任务 8：统计 1 到 359516 的素数个数。
[Worker 1] 开始处理任务 1，计算 1 到 486651 的素数个数。
[Parent] 投放任务 9：统计 1 到 321253 的素数个数。
[Worker 1] 完成任务 1，耗时 44 ms。
[Worker 1] 开始处理任务 2，计算 1 到 425457 的素数个数。
[Parent] 投放任务 10：统计 1 到 429394 的素数个数。
[Worker 1] 完成任务 2，耗时 32 ms。
[Worker 1] 开始处理任务 3，计算 1 到 336876 的素数个数。
[Parent] 投放任务 11：统计 1 到 361557 的素数个数。
[Worker 1] 完成任务 3，耗时 24 ms。
[Worker 1] 开始处理任务 4，计算 1 到 316460 的素数个数。
[Parent] 投放任务 12：统计 1 到 308749 的素数个数。
[Worker 1] 完成任务 4，耗时 22 ms。
[Worker 1] 开始处理任务 5，计算 1 到 359361 的素数个数。
[Parent] 投放任务 13：统计 1 到 301548 的素数个数。
[Worker 1] 完成任务 5，耗时 27 ms。
[Worker 1] 开始处理任务 6，计算 1 到 499607 的素数个数。
[Parent] 投放任务 14：统计 1 到 488501 的素数个数。
[Worker 1] 完成任务 6，耗时 42 ms。
[Worker 1] 开始处理任务 7，计算 1 到 484846 的素数个数。
[Worker 1] 完成任务 7，耗时 37 ms。
[Worker 1] 开始处理任务 8，计算 1 到 359516 的素数个数。
[Worker 1] 完成任务 8，耗时 25 ms。
[Worker 1] 开始处理任务 9，计算 1 到 321253 的素数个数。
[Worker 1] 完成任务 9，耗时 21 ms。
[Worker 1] 开始处理任务 10，计算 1 到 429394 的素数个数。
[Parent] 投放任务 15：统计 1 到 413890 的素数个数。
[Parent] 投放任务 16：统计 1 到 480608 的素数个数。
[Parent] 投放任务 17：统计 1 到 471208 的素数个数。
[Parent] 投放任务 18：统计 1 到 350106 的素数个数。
[Worker 1] 完成任务 10，耗时 39 ms。
[Worker 1] 开始处理任务 11，计算 1 到 361557 的素数个数。
[Parent] 投放任务 19：统计 1 到 457059 的素数个数。
```

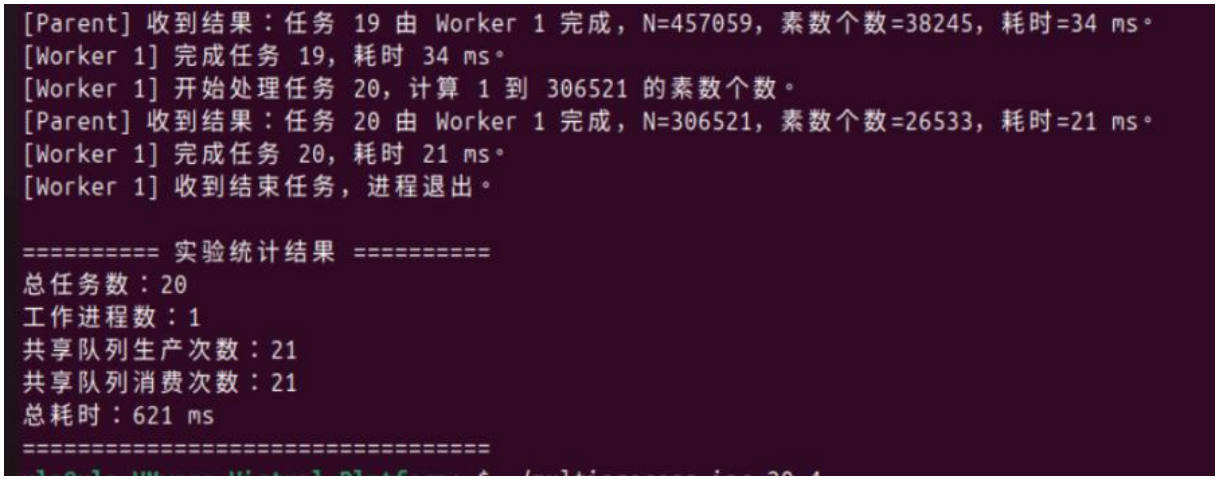


图 2 单进程运行结果

8.2 四进程运行结果

当任务数量为 20，工作进程数量为 4 时，程序运行结果如下：

```
==================================
xln@xln-VMware-Virtual-Platform:~$ ./multiprocess_ipc 20 4
========== 多进程 IPC 实验 ==========
任务数量：20
工作进程数量：4
队列容量：8
==================================
[Parent] 投放任务 1：统计 1 到 408842 的素数个数。
[Parent] 投放任务 2：统计 1 到 414398 的素数个数。
[Parent] 投放任务 3：统计 1 到 439731 的素数个数。
[Parent] 投放任务 4：统计 1 到 480248 的素数个数。
[Parent] 投放任务 5：统计 1 到 338929 的素数个数。
[Parent] 投放任务 6：统计 1 到 426700 的素数个数。
[Parent] 投放任务 7：统计 1 到 312637 的素数个数。
[Parent] 投放任务 8：统计 1 到 482468 的素数个数。
[Parent] 投放任务 9：统计 1 到 433589 的素数个数。
[Worker 4] 开始处理任务 1，计算 1 到 408842 的素数个数。
[Worker 2] 开始处理任务 3，计算 1 到 439731 的素数个数。
[Worker 1] 开始处理任务 2，计算 1 到 414398 的素数个数。
[Parent] 投放任务 10：统计 1 到 359117 的素数个数。
[Parent] 投放任务 11：统计 1 到 414680 的素数个数。
[Parent] 投放任务 12：统计 1 到 423957 的素数个数。
[Worker 3] 开始处理任务 4，计算 1 到 480248 的素数个数。
[Worker 1] 完成任务 2，耗时 60 ms。
[Parent] 投放任务 13：统计 1 到 349664 的素数个数。
[Worker 1] 开始处理任务 5，计算 1 到 338929 的素数个数。
[Worker 2] 完成任务 3，耗时 69 ms。
[Parent] 投放任务 14：统计 1 到 390905 的素数个数。
[Worker 2] 开始处理任务 6，计算 1 到 426700 的素数个数。
[Worker 4] 完成任务 1，耗时 92 ms。
[Worker 4] 开始处理任务 7，计算 1 到 312637 的素数个数。
[Parent] 投放任务 15：统计 1 到 354371 的素数个数。
[Worker 1] 完成任务 5，耗时 45 ms。
[Parent] 投放任务 16：统计 1 到 331835 的素数个数。
[Worker 1] 开始处理任务 8，计算 1 到 482468 的素数个数。
[Worker 3] 完成任务 4，耗时 112 ms。
[Worker 3] 开始处理任务 9，计算 1 到 433589 的素数个数。
```

```
[Worker 3] 开始处理任务 17，计算 1 到 414331 的素数个数。
[Parent] 收到结果：任务 14 由 Worker 3 完成，N=390905，素数个数=33142，耗时=70 ms。
[Parent] 收到结果：任务 12 由 Worker 2 完成，N=423957，素数个数=35694，耗时=83 ms。
[Worker 2] 完成任务 12，耗时 83 ms。
[Worker 2] 开始处理任务 18，计算 1 到 443189 的素数个数。
[Worker 4] 完成任务 15，耗时 62 ms。
[Parent] 收到结果：任务 15 由 Worker 4 完成，N=354371，素数个数=30325，耗时=62 ms。
[Worker 4] 开始处理任务 19，计算 1 到 324768 的素数个数。
[Parent] 收到结果：任务 16 由 Worker 1 完成，N=331835，素数个数=28550，耗时=54 ms。
[Worker 1] 完成任务 16，耗时 54 ms。
[Worker 1] 开始处理任务 20，计算 1 到 424525 的素数个数。
[Worker 4] 完成任务 19，耗时 32 ms。
[Worker 4] 收到结束任务，进程退出。
[Parent] 收到结果：任务 19 由 Worker 4 完成，N=324768，素数个数=27984，耗时=32 ms。
[Parent] 收到结果：任务 18 由 Worker 2 完成，N=443189，素数个数=37196，耗时=58 ms。
[Worker 2] 完成任务 18，耗时 58 ms。
[Worker 2] 收到结束任务，进程退出。
[Worker 3] 完成任务 17，耗时 73 ms。
[Worker 3] 收到结束任务，进程退出。
[Parent] 收到结果：任务 17 由 Worker 3 完成，N=414331，素数个数=34941，耗时=73 ms。
[Parent] 收到结果：任务 20 由 Worker 1 完成，N=424525，素数个数=35740，耗时=52 ms。
[Worker 1] 完成任务 20，耗时 52 ms。
[Worker 1] 收到结束任务，进程退出。

========== 实验统计结果 ==========
总任务数：20
工作进程数：4
共享队列生产次数：24
共享队列消费次数：24
总耗时：378 ms
==================================
xln@xln-VMware-Virtual-Platform:~$ ^C
xln@xln-VMware-Virtual-Platform:~$
```

图 3 四进程运行结果

8.3 实验结果对比

实验编号	任务数量	工作进程数	共享队列生产次数	共享队列消费次数	总耗时
1	20	1	21	21	621 ms
2	20	4	24	24	378 ms

由实验结果可以看出，当工作进程数量由 1 个增加到 4 个时，程序总耗时由 621 ms 降低到 378 ms。

加速比 = 单进程耗时 / 四进程耗时 = 621 / 378 ≈ 1.64。

耗时降低比例 = (621 - 378) / 621 × 100% ≈ 39.13%。因此，在本实验任务规模下，使用 4 个工作进程相较于使用 1 个工作进程，程序运行时间明显降低，体现了多进程并发处理的优势。

九、结果分析

9.1 共享队列生产和消费次数分析

在单进程实验中，总任务数为 20，但共享队列生产次数和消费次数均为 21。这是因为除了 20 个正式计算任务外，父进程还额外投放了 1 个结束任务，用于通知唯一的工作进程退出。因此，生产次数和消费次数均为 $20 + 1 = 21$ 次。

在四进程实验中，总任务数为 20，但共享队列生产次数和消费次数均为 24。这是因为除了 20 个正式计算任务外，父进程还需要为 4 个工作进程分别投放一个结束任务。因此，生产次数和消费次数均为 $20 + 4 = 24$ 次。由此可知，两组实验中的生产次数和消费次数均符合程序设计逻辑，说明共享队列的任务投放和任务消费过程是正确的。

9.2 多进程并发优势分析

在单进程情况下，所有任务只能由一个工作进程顺序处理。虽然程序仍然使用共享内存和信号量，但任务执行过程本质上是串行的，因此总耗时较长。在四进程情况下，多个子进程可以同时从共享队列中取出任务并执行素数统计。操作系统调度多个进程并发运行，使任务可以被分配到不同工作进程中处理，从而缩短总运行时间。

实验数据显示，4 个工作进程的总耗时为 378 ms，明显低于 1 个工作进程的 621 ms。这说明在计算任务规模较大时，多进程并发能够有效提高程序执行效率。

9.3 同步互斥机制分析

由于多个进程需要同时访问共享任务队列，如果不设置同步互斥机制，就可能出现多个进程同时读取同一个任务、父进程覆盖尚未被处理的任务、队列 in 和 out 指针混乱、任务数量统计错误等问题。本实验通过 sem_empty、sem_full 和 sem_mutex 三个信号量解决了这些问题。

其中，sem_empty 保证父进程不会在队列满时继续写入任务，sem_full 保证子进程不会在队列空时读取任务，sem_mutex 保证同一时刻只有一个进程可以访问共享队列的临界区。从实验结果看，生产次数和消费次数完全对应，说明同步互斥机制工作正常。

9.4 进程间通信机制分析

本实验同时使用了共享内存和管道两种进程间通信方式。共享内存用于父进程向子进程分发任务，管道用于子进程向父进程返回计算结果。共享内存适合传递共享任务队

列，因为其通信效率较高，多个进程可以直接访问同一块内存区域；管道适合传递子进程处理完成后的结果，因为结果数据量较小，且传输方向明确。

十、实验中遇到的问题及解决方法

10.1 初始任务规模较小导致多进程优势不明显

在实验初期，如果设置的素数统计范围较小，单个任务计算时间较短。此时，多进程程序中的进程创建、信号量同步、共享内存访问和管道通信等开销占比较大，可能导致多进程运行时间不一定优于单进程。解决方法是适当增大每个任务的计算规模，使任务成为计算密集型任务。修改后，4 个工作进程的运行耗时明显低于 1 个工作进程，能够更好地体现多进程并发优势。

10.2 共享队列访问需要避免数据竞争

多个子进程可能同时访问共享任务队列，如果不加互斥保护，会造成任务重复读取或队列指针错误。本实验通过 sem_mutex 信号量保护临界区，使同一时刻只有一个进程能够修改共享队列。

10.3 子进程退出需要结束任务

如果父进程只投放正式任务而不投放结束任务，子进程在处理完任务后会继续等待新任务，程序无法正常结束。因此，父进程在所有正式任务投放完成后，为每个工作进程投放一个结束任务。当子进程读取到结束任务后主动退出。

十一、实验总结

通过本次实验，完成了一个 Linux 下基于共享内存、信号量和管道的多进程并发任务处理系统。实验中，父进程负责产生素数统计任务，多个子进程负责并发处理任务，子进程完成计算后通过管道将结果返回给父进程。

本实验较好地体现了操作系统中的进程创建、进程间通信、同步互斥和并发执行机制。通过共享内存实现任务队列共享，通过信号量保证共享队列访问的正确性，通过管道实现计算结果回传。

实验结果表明，在任务数量为 20 的情况下，1 个工作进程运行耗时为 621 ms，4 个工作进程运行耗时为 378 ms。4 个工作进程相较于 1 个工作进程，加速比约为 1.64，耗时降低约 39.13%。这说明在计算任务规模较大时，多进程并发处理可以有效提高程序执行效率。

同时，本实验也说明，并发程序并不是在任何情况下都会更快。当任务规模较小时，进程调度、通信和同步开销可能抵消并发带来的优势。因此，在实际系统设计中，需要根据任务类型、计算规模和通信开销合理选择并发模型。通过本次实验，我进一步理解了 Linux 操作系统中进程管理和进程间通信机制的实际应用，提高了使用 C 语言进行系统级程序设计的能力。

附录：主要源程序

以下为本实验的主要源程序。

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <sys/wait.h>
#include <semaphore.h>
#include <string.h>
#include <errno.h>
#include <time.h>

#define QUEUE_SIZE 8

#define SHM_NAME "/os_exp_shared_queue"
#define SEM_EMPTY "/os_exp_sem_empty"
#define SEM_FULL    "/os_exp_sem_full"
#define SEM_MUTEX "/os_exp_sem_mutex"

typedef struct {
    int id;
    int number;
} task_t;

typedef struct {
    task_t queue[QUEUE_SIZE];
    int in;
    int out;
    int produced;
    int consumed;
} shared_queue_t;

typedef struct {
    int worker_id;
    int task_id;
    int number;
    int prime_count;
    long elapsed_ms;
} result_t;
```

```
long current_time_ms() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1000 + ts.tv_nsec / 1000000;
}

int is_prime(int n) {
    if (n < 2) {
        return 0;
    }
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) {
            return 0;
        }
    }

    return 1;
}

int count_primes(int n) {
    int count = 0;

    for (int i = 2; i <= n; i++) {
        if (is_prime(i)) {
            count++;
        }
    }

    return count;
}

void cleanup_ipc_objects() {
    shm_unlink(SHM_NAME);
    sem_unlink(SEM_EMPTY);
    sem_unlink(SEM_FULL);
    sem_unlink(SEM_MUTEX);
}

void put_task(shared_queue_t *shared_queue,
              sem_t *sem_empty,
              sem_t *sem_full,
              sem_t *sem_mutex,
              task_t task) {
```

```
    sem_wait(sem_empty);
    sem_wait(sem_mutex);

    shared_queue->queue[shared_queue->in] = task;
    shared_queue->in = (shared_queue->in + 1) % QUEUE_SIZE;
    shared_queue->produced++;

    sem_post(sem_mutex);
    sem_post(sem_full);
}

task_t get_task(shared_queue_t *shared_queue,
                sem_t *sem_empty,
                sem_t *sem_full,
                sem_t *sem_mutex) {
    task_t task;

    sem_wait(sem_full);
    sem_wait(sem_mutex);

    task = shared_queue->queue[shared_queue->out];
    shared_queue->out = (shared_queue->out + 1) % QUEUE_SIZE;
    shared_queue->consumed++;
    sem_post(sem_mutex);
    sem_post(sem_empty);

    return task;
}

void worker_process(int worker_id,
                    shared_queue_t *shared_queue,
                    sem_t *sem_empty,
                    sem_t *sem_full,
                    sem_t *sem_mutex,
                    int pipe_write_fd) {
    while (1) {
        task_t task = get_task(shared_queue, sem_empty, sem_full, sem_mutex);

        if (task.id == -1) {
            printf("[Worker %d] 收到结束任务，进程退出。\n", worker_id);
            break;
        }

        printf("[Worker %d] 开始处理任务 %d，计算 1 到 %d 的素数个数。\n",
```

```
                worker_id, task.id, task.number);

        long start = current_time_ms();
        int prime_count = count_primes(task.number);
        long end = current_time_ms();

        result_t result;
        result.worker_id = worker_id;
        result.task_id = task.id;
        result.number = task.number;
        result.prime_count = prime_count;
        result.elapsed_ms = end - start;

        write(pipe_write_fd, &result, sizeof(result));

        printf("[Worker %d] 完成任务 %d，耗时 %ld ms。\n",
                worker_id, task.id, result.elapsed_ms);
    }

    close(pipe_write_fd);
    exit(0);
}

int main(int argc, char *argv[]) {
    setbuf(stdout, NULL);

    int task_count = 20;
    int worker_count = 4;

    if (argc >= 2) {
        task_count = atoi(argv[1]);
    }

    if (argc >= 3) {
        worker_count = atoi(argv[2]);
    }

    if (task_count <= 0) {
        task_count = 20;
    }

    if (worker_count <= 0 || worker_count > 8) {
        worker_count = 4;
    }
```

```
    printf("========== 多进程 IPC 实验 ==========\n");
    printf("任务数量：%d\n", task_count);
    printf("工作进程数量：%d\n", worker_count);
    printf("队列容量：%d\n", QUEUE_SIZE);
    printf("====================================\n");

    cleanup_ipc_objects();

    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1) {
        perror("shm_open error");
        exit(1);
    }

    if (ftruncate(shm_fd, sizeof(shared_queue_t)) == -1) {
        perror("ftruncate error");
        exit(1);
    }

    shared_queue_t *shared_queue = mmap(NULL,
                                        sizeof(shared_queue_t),
                                        PROT_READ | PROT_WRITE,
                                        MAP_SHARED,
                                        shm_fd,
                                        0);

    if (shared_queue == MAP_FAILED) {
        perror("mmap error");
        exit(1);
    }

    memset(shared_queue, 0, sizeof(shared_queue_t));

    sem_t *sem_empty = sem_open(SEM_EMPTY, O_CREAT, 0666, QUEUE_SIZE);
    sem_t *sem_full = sem_open(SEM_FULL, O_CREAT, 0666, 0);
    sem_t *sem_mutex = sem_open(SEM_MUTEX, O_CREAT, 0666, 1);

    if (sem_empty == SEM_FAILED || sem_full == SEM_FAILED || sem_mutex ==
SEM_FAILED) {
        perror("sem_open error");
        cleanup_ipc_objects();
        exit(1);
    }
```

```
int result_pipe[2];

if (pipe(result_pipe) == -1) {
        perror("pipe error");
        cleanup_ipc_objects();
        exit(1);
}

pid_t workers[8];

for (int i = 0; i < worker_count; i++) {
        pid_t pid = fork();

        if (pid < 0) {
                perror("fork error");
                cleanup_ipc_objects();
                exit(1);
        }

        if (pid == 0) {
                close(result_pipe[0]);
                worker_process(i + 1,
                                shared_queue,
                                sem_empty,
                                sem_full,
                                sem_mutex,
                                result_pipe[1]);
        } else {
                workers[i] = pid;
        }
}

close(result_pipe[1]);

srand(1);

long total_start = current_time_ms();

for (int i = 0; i < task_count; i++) {
        task_t task;
        task.id = i + 1;
        task.number = 300000 + rand() % 200000;
```

```
            put_task(shared_queue, sem_empty, sem_full, sem_mutex, task);

            printf("[Parent] 投放任务 %d：统计 1 到 %d 的素数个数。\n",
                   task.id, task.number);
        }

        for (int i = 0; i < worker_count; i++) {
            task_t stop_task;
            stop_task.id = -1;
            stop_task.number = 0;

            put_task(shared_queue, sem_empty, sem_full, sem_mutex, stop_task);
        }

        int received = 0;

        while (received < task_count) {
            result_t result;
            ssize_t bytes = read(result_pipe[0], &result, sizeof(result));

            if (bytes == sizeof(result)) {
                printf("[Parent] 收到结果：任务 %d 由 Worker %d 完成，N=%d，素数个数=%d，耗时=%ld ms。\n",
                       result.task_id,
                       result.worker_id,
                       result.number,
                       result.prime_count,
                       result.elapsed_ms);

                received++;
            }
        }

        long total_end = current_time_ms();

        for (int i = 0; i < worker_count; i++) {
            waitpid(workers[i], NULL, 0);
        }

        printf("\n========== 实验统计结果 ==========\n");
        printf("总任务数：%d\n", task_count);
        printf("工作进程数：%d\n", worker_count);
        printf("共享队列生产次数：%d\n", shared_queue->produced);
        printf("共享队列消费次数：%d\n", shared_queue->consumed);
```

```
    printf("总耗时：%ld ms\n", total_end - total_start);
    printf("==================================\n");

    close(result_pipe[0]);

    sem_close(sem_empty);
    sem_close(sem_full);
    sem_close(sem_mutex);

    munmap(shared_queue, sizeof(shared_queue_t));
    close(shm_fd);

    cleanup_ipc_objects();

    return 0;
}
```